

# FRED TALKS

17<sup>th</sup> September 2026

---

## CONTENTS

“Code was never the hard part” is an insult to all programmers

SENKO RAŠIĆ · 1400 WORDS

---

Building agents that reach production systems with MCP

CLAUDE.COM · 1746 WORDS

---

Mastering Codex Remote for engineering

DEVELOPERS.OPENAI.COM · 2207 WORDS

---

# “Code was never the hard part” is an insult to all programmers

SENKO RAŠIĆ · 09 AUG 2026 · [SOURCE](#)

---

The software development profession is in the midst of upheaval. Nobody knows how the AI revolution will play out in the end, but it is clear many aspects of work and life will be transformed—including programming.

One of the comments I hear often lately boils down to “*LLMs may be good at coding, but software was never the hard part*” and “*coding is easy, it's figuring out what to code that's hard*”.

I believe that's a gross insult to all programmers everywhere.

## If coding is easy...

If coding is easy, how come programmers were in high demand, and have demanded large salaries for years (even before ZIRP)? Why was there so much stress, overwork and burnout even before AI started churning out 5000-line PRs? Why did companies seek 10x ninja rockstar coders and subject them to leetcode interviews—surely, a junior fresh out of college could churn out something if it's so easy?

If coding is easy, why do we have doorstoppers like [Clean Code](#) and [The Pragmatic Programmer](#)? Is [The Art of Computer Programming](#) a light summer read? Is SICP a coffee-table book? Why do we have bootcamps or even whole college degrees dedicated to it?

If coding is easy, was [Carmack](#) just at the right place at the right time? Why do we consider [Fabrice Bellard](#) a genius?

If coding is easy, why are people angry at AI (or anyone else) copying their code? Why do they act like they've poured their sweat, soul, and copious amounts of time into something so trivial?

If coding is easy, why do many now feel like their identity and professional purpose are being stripped away from them?

If coding is easy, why is software so damn buggy?

# If figuring out what to build is the hard part...

If deciding what to build is the hard part, why do so many product managers seem clueless? Why aren't there rigorous 10-step interviews for them? Why aren't they getting paid more than the developers?

If deciding what to build is the hard part, why aren't market researchers, usability experts and—hell, customer success—considered rockstars in a software company? If “understanding the customer” is harder, why are business analysts looked down on as pencil pushers?

If implementation is easy and finding demand is harder, why are programmers upset when the salespeople promise a new feature to a customer to close the sale? They've found a genuine demand, something people will pay for!

If coding is easy, why doesn't everyone just build ten variations of a thing and see which pans out?

## There's no median programmer

Another cliché comment is *“most work in software development is talking to stakeholders, understanding the customer's needs, and having clarity on the priorities”*.

I have met many programmers throughout my career, and very few of them want to talk to stakeholders, much less customers (exceptions are freelancers and founders, especially of software development shops). And, “having clarity on the priorities” boils down to “just tell me what to do and don't switch it up every two days”.

Some software developers do say “I don't write code, I solve customer's problems”. But then they turn around and start to opine on monads, memory safety, and DRY principles, while their understanding of the customer is a made-up “user persona”, and they think “affordance” is the money your parents used to give you on weekends so you could go out and have a good time.

Yet others will say “Software development is theory building”. Programs are actually proofs (as in, mathematical proofs). Every commit should tell a story. And solving a customer's problem by FTPing a PHP file is a cardinal sin.

I don't mean to imply there are no developers that simultaneously care deeply about the craft of software development and really empathize with the customer. I do believe they might want to see a professional about a split personality disorder, tho.

## What *is* important?

I do believe that talking to users, understanding their experience, empathizing with them, solving customers' problems and having all the stakeholders on the same page is critical to the success of a software project.

I also believe that creating good code is a craft that requires skill, patience, attention to detail, experience and wisdom, and that it will continue to be relevant in the times ahead.

¿Por qué no los dos?

To the extent that we can pull it off, I think we should aim for both. A deep understanding of the system we're building, together with a deep understanding of *why* we're building it.

Loudly proclaiming that “code is easy” or, at the opposite end, “code is art, a creative human expression that cannot be automated”, is just burying our heads in the sand.

It's cope. And you don't want cope, you want to thrive.

By this, I don't mean “jump on the LLM bandwagon.” I don't mean “become a manager of fleets of AI agents.” I also don't mean “AI-generated code is stolen slop garbage, fight it with tooth and nail, the bubble will pop soon enough anyways.”

But do recognize we're in the middle of an industry-wide tectonic change. We need to figure out how to adapt. We need to understand what is likely to change and what never changes.

## What doesn't change?

Software will be getting more complex. Software will always need maintenance: bit-rot is a fact of life. So is entropy. Technology (hardware and software) will move forward, for better or worse. The tower (skyscraper?) of abstractions grows ever higher.

Users will always want more and be prepared to spend less. They still won't know how to relay their needs and wants. Worse, they still won't know exactly what they want. The disconnect between the customers (who actually pay for the software) and users (who use it) will still be here, as will the tension between the needs of the business and the needs of its customers.

Also: there will never be a shortage of snake oil salesmen. Tech du jour comes and goes (I'm still waiting for the new VR renaissance!)

# What changes?

Programmers have been in the business of disrupting our own industry since the beginning. Nobody uses punch-cards any more. Very few people need to code in assembly, or COBOL. Those decades spent fighting memory bugs in C or C++, with the scars to prove it, are worthless in the age of Rust, Go, Python and JavaScript.

I'm old enough to appreciate valgrind or remember `mysql_real_escape_string()` from the PHP4 era—stuff I'll never again need in my life. And that wasn't even so long ago! I narrowly missed the dBase, Clipper, HyperCard and Access era, technologies which I can still spot operating in shops, cafes, or a dusty, once beige and now golden-brown, midi-tower still happily running some bespoke biz solution (backups? what backups?)

# How do we thrive?

Accept that change happens. Be equal parts curious and critical about the new stuff.

Understand there's a lot of hype and try to discriminate between hot air and what really works (and to what extent). Also be aware of ever-shifting goalposts: stand back and look at the past year, or five, and assess the velocity of change (technical, economic, societal).

Your role and your responsibilities *will* be changing. Be willing to invest time and energy into better understanding fields or roles adjacent to yours.

If you're a senior developer, don't just find solace in deepening your expertise. Learn about user experience, customer interviews, or business strategies for the companies in your domain. It will help you gain a better appreciation of all the work done to put a piece of software into users' hands, whether or not you'll actually ever have to do any of those other bits.

If you're just starting or are junior in your role: invest in deepening your understanding of how software works. Understanding pointers, recursion, or memory hierarchy will help you even if you're a JavaScript developer. Understanding network protocols and how HTTP works will be useful even if you're building WordPress plugins. Do leetcode and learn about algorithms and data structures even if you don't need to. Don't be afraid to ask *why* and *how exactly*.

For inspiration, here are a few books and other resources that might be helpful:

- [Continuous Discovery Habits](#)

# One more thing

Whoever you are, don't outsource your understanding, judgement, empathy and taste to AI. Don't abdicate your responsibility. Don't be a meat proxy.

---

## Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · [SOURCE](#)

---

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

## Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

### Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the  $M \times N$  integration problem.

## Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it's thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

## Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

## Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We're already seeing this in adoption. The [MCP SDKs](#) recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including [Claude Cowork](#), [Claude Managed Agents](#), and [channels in Claude Code](#).

As MCP continues to support production agentic systems, we're sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

# Building effective MCP servers

We have over 200 MCP servers in our [directory](#), used by millions of people every day. From working closely with enterprises and developers building on the protocol, we've spotted a handful of design patterns that determine how reliably agents can use a server.

## Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

## Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single `create_issue_from_thread` tool beats `get_thread` + `parse_messages` + `create_issue` + `link_attachment`. See [writing effective tools for agents](#) to learn more about the full pattern.

## Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. [Cloudflare's MCP server](#) is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

## Ship rich semantics where they help

[MCP Apps](#) is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

[Elicitation](#) lets your server pause mid-tool call to ask the user for input. [Form mode](#) sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. [URL mode](#) hands the user to a browser—

use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

## Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest [MCP spec](#) supports [CIMD](#) (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. [Vaults in Claude Managed Agents](#) covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

## Making MCP clients more context-efficient

MCP standardizes how AI agents ([clients](#)) connect to and work with tools and data sources they need ([servers](#)). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

## Load tool definitions on demand with tool search

[Tool search](#) defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our [testing](#), tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: [advanced tool use](#)

## Process tool results in code with programmatic tool calling

Programmatic tool calling processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly 37% on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See [advanced tool use](#) for the full breakdown.

## Pairing MCP servers with skills

Skills and MCP are complementary. MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

### Bundle skills and MCP servers as a plugin

Plugins for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our [data plugin](#) for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: [Extending Claude's capabilities with skills and MCP servers](#)

### Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. [Canva](#), [Notion](#), [Sentry](#), and more do this today in Claude, listing the skill next to their connector in our [web directory](#).

To make that pairing portable across every client, the MCP community is actively working on an extension for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

## The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

## Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

---

# Mastering Codex Remote for engineering

DEVELOPERS.OPENAI.COM · 25 JUN 2026 · [SOURCE](#)

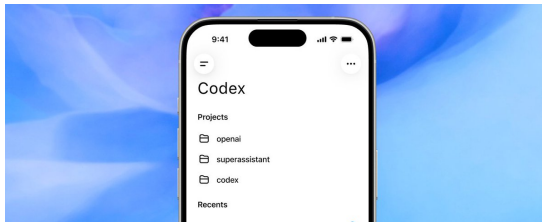
---

Codex Remote is easy to underestimate.

At first glance, it looks like a way to check on a coding task from your phone. That is useful, but it misses the bigger idea. The real power of Codex Remote is that it lets you start, direct, review, and organize work running on your development machines without pretending that an iPhone should be a tiny terminal.

Over the last two months, we have added a surprising amount of depth to that loop: remote host connections, worktrees, goals, side chats, inline code review, queued and steering prompts, attachments, skills and plugins, archived threads, security controls, and a long tail of small details that make the app useful for serious work.

This is the field guide I wish every new power user had.



## The right mental model: Your phone is the control plane

The code still runs where it belongs: on your Mac, Windows machine, devbox, or other connected host. Codex Remote gives you a native interface for controlling that work.

That distinction matters. The goal is not to reproduce every terminal affordance on a small screen. The goal is to make the decisions that unblock an agent easy from anywhere:

- What repository and workspace should it use?
- Should this run in the current branch or a fresh worktree?
- Should my next message wait, or should it redirect the active turn?
- Is this command safe to approve?
- What changed, and do I agree with it?
- Should this become a durable goal, a separate thread, or a quick side question?

Once you use the app this way, it stops feeling like remote desktop software and starts feeling like an engineering control plane.

## 1. Start the task with the right boundaries

Good agent work starts with a correctly scoped environment. Codex Remote lets you choose the connected host and workspace before the first prompt. For a new thread, you can also choose a branch, create a separate worktree, and run the environment setup associated with it.

That makes a few useful patterns possible:

- Use the current checkout for a quick investigation.
- Create a new worktree for a change that should stay isolated.
- Start from the intended base branch instead of fixing Git state later.
- Let the environment setup run before asking Codex to build or test.

This is one of the most important power-user habits: Spend 10 seconds choosing the right execution context and save 10 minutes of cleanup later.

The composer can also carry more context than plain text. You can attach files, photos, or a fresh camera capture. Skills and plugins appear inline, which makes it much easier to confirm that your prompt is invoking the capability you intended.

My rule is simple: If a screenshot, file, or named skill can remove ambiguity, attach or mention it before the first turn.

## 2. Learn the difference between Queue and Steer

This is probably the least obvious high-leverage setting in Codex Remote.

When Codex is already working, a follow-up can do one of two things:

- **Queue** waits until the current response finishes, then sends your prompt as the next turn.
- **Steer** injects guidance into the work already in progress.

Queue is the safe default. Use it for a second task, an extra test request, or anything that should happen after the current work.

Steer is for correcting direction while the cost of continuing down the wrong path is still growing:

- | Keep the fix inside the mobile package. Do not refactor the shared renderer.
- | The reproduction only happens after reconnect. Test the resumed path, not the live path.
- | Stop investigating the UI. Check whether the server removed the item during resume.

That turns the phone into something more useful than a status screen. You can intervene at the exact moment a run needs judgment.

You can choose the default follow-up behavior in Codex settings. I keep Queue as the default and use Steer deliberately; accidental mid-turn redirection is usually more expensive than waiting.

### 3. Use side chats as a branch of thought

Long-running coding threads accumulate valuable context. Interrupting one with every side question makes the main transcript noisy and can pull the agent away from the objective.

That is what side chats solve.

Use `/side` to open a lightweight conversation connected to the current thread. Use `/side <prompt>` to open it with a question already prepared. Even better, select text in the transcript and choose **Ask in side chat**. The selected passage becomes the starting context for the new conversation.

I use side chats for questions such as:

- Why did Codex choose this architecture?
- What does this error actually mean?
- Is this behavior consistent with the desktop app?
- Give me a release-note version of this implementation detail.
- What should I verify before I approve this command?

The distinction is useful: The main thread owns the work; the side chat helps me understand the work.

## 4. Use Plan for the path and Goal for the outcome

Plan mode and goals solve different problems.

Plan mode asks Codex to propose the implementation path before changing code. It is useful when the task is underspecified, risky, or likely to touch several systems.

A goal is durable. It tells Codex what outcome to keep pursuing across turns. On mobile, `/goal` can create and manage that objective, while progress remains visible as the work continues.

A practical pattern is:

1. Start with Plan mode for a risky change.
2. Inspect the proposed boundaries.
3. Turn the accepted outcome into a goal when the work will require multiple iterations.
4. Let Codex continue through implementation, tests, review feedback, and cleanup without restating the objective every time.

Plans answer “How should we approach this?” Goals answer “What must be true before we are done?”

## 5. Review code without leaving the conversation

The review loop is where Codex Remote becomes genuinely useful for engineering rather than merely convenient.

Completed turns can show a summary of changed files. From there, you can open the diff, inspect individual files, expand or collapse sections, wrap long lines, and open a source file with syntax highlighting. You can attach inline comments to the relevant lines and send that review context back to Codex.

There are several levels to the workflow:

- Open the changed-file summary for a quick sanity check.

- Tap through to the full source when a diff lacks surrounding context.
- Add inline comments for precise corrections.
- Use the review command to review local changes or compare against a branch.
- Link a file back into the chat when you want Codex to reason about that file specifically.

This makes a tight mobile review loop possible:

1. Codex completes the implementation.
2. I inspect the diff from my phone.
3. I leave two inline comments.
4. Codex addresses them in the same thread.
5. I review the smaller follow-up diff.

The important point is not that a phone replaces a large monitor for deep code reading. It does not. The point is that many reviews are blocked on one or two decisions, and those decisions no longer need to wait until I am back at my desk.

## 6. Treat permissions as part of the workflow

Remote work is only useful if control stays explicit.

Codex Remote surfaces approval requests for commands, file changes, network access, and connected tools. Depending on the request and host configuration, an approval may apply once, for the current chat, or more broadly.

The power-user move is not to approve everything. It is to choose the narrowest permission that keeps the work moving.

For a well-understood command in a trusted thread, a chat-scoped approval can remove repetitive interruptions. For an unfamiliar command, a sensitive repository, or a request whose effect is unclear, approve once—or deny it and ask Codex to explain or use a safer approach.

You can also choose the thread's broader approval behavior when starting work. Think of this as part of task setup, alongside the host, workspace, branch, and model.

## 7. Manage context before it becomes a problem

Agent threads are stateful, and long threads eventually accumulate enough context to become slower or less focused.

Codex Remote has a small set of tools for managing that lifecycle:

- `/status` shows session details, workspace, context usage, and available rate-limit information.
- The optional context indicator keeps remaining context visible in the composer.
- `/compact` compresses an overgrown thread while preserving the useful working state.
- `/fork` creates a new thread from the current one when you want shared history but a different direction.

The practical sequence is: Check status, compact when the objective is still the same, and fork when the objective has diverged.

Do not use a side chat and a fork interchangeably. A side chat is a lightweight question around the current work. A fork is a new primary line of work that inherits the thread's history.

## 8. Keep a clean Thread Desk

My own Codex workflow increasingly looks like a small operations desk.

I pin the few threads that are actively moving, rename them around outcomes, and archive them aggressively once the work is done. Archived-thread browsing makes this safe: Archive is organization, not deletion.

Notifications are part of that system. A completion notification can open the relevant Codex task directly, so the handoff from “agent finished” to “human reviews” is one tap.

Spotlight and Shortcuts can open Codex Remote directly. On iPad, keyboard shortcuts make the app surprisingly efficient: Create a new thread, move between threads, open changed files, pin, rename, and archive without reaching for every control.

This is the “chief of staff” use case for Codex Remote. The app is not only where I issue prompts. It is where I keep track of which pieces of engineering work are active, blocked, waiting for review, or finished.

## 9. The hidden command palette

Typing `/` reveals the fastest route to many of the app's advanced behaviors. Availability can depend on the connected host, app version, and account configuration.

| Command                              | What it is for                                               |
|--------------------------------------|--------------------------------------------------------------|
| <code>/plan</code>                   | Toggle Plan mode before implementation.                      |
| <code>/goal &lt;objective&gt;</code> | Create or update a durable objective.                        |
| <code>/side [question]</code>        | Ask a side question without disrupting the main thread.      |
| <code>/review</code>                 | Review local changes or compare them with a branch.          |
| <code>/status</code>                 | Inspect the session, workspace, context, and rate limits.    |
| <code>/compact</code>                | Compress a long thread's context.                            |
| <code>/fork</code>                   | Create a new primary thread from the current history.        |
| <code>/fast</code>                   | Switch between standard and faster execution when available. |
| <code>/feedback</code>               | Send product feedback tied to the current session.           |

The command palette is worth learning because it exposes the product's real conceptual model: Plan, pursue, branch, review, inspect, and recover.

## 10. Five workflows that work especially well on mobile

### The release captain

Start a focused thread for a release or pull request. Ask Codex to check the current branch, CI state, open review feedback, and release inclusion. Pin the thread. When new information arrives, steer only when it invalidates the active investigation; otherwise, queue it. Review the final diff or release note from the phone and archive the thread when the release lands.

### The interrupt-driven bug fix

Attach a screenshot, log, or captured file. Ask Codex to diagnose before editing. Use a side chat to interrogate one suspicious error without derailing the main investigation. Once the cause is clear, return to the main thread and authorize the narrow fix.

## The mobile reviewer

Run a review against the intended branch, inspect the changed-file summary, open the files that matter, and attach inline comments. Ask Codex to address only those comments, then review the follow-up diff.

## The long-running objective

Create a goal with a concrete completion condition: Tests green, review feedback resolved, or a reproducible performance threshold met. Check progress from notifications and status rather than repeatedly asking “Are you done?” Use queued prompts for additional work and reserve steering for corrections.

## The multi-machine operator

Keep hosts named clearly and organize work by machine and workspace. Use the phone to start a task on the machine that has the right checkout, credentials, simulator, or operating system. This is especially useful when one task requires a Mac and another needs a Windows host.

## Small features with outsized impact

Some of my favorite additions are not headline features:

- Edit the latest sent prompt instead of adding a correction turn.
- Save or copy rendered images from a task.
- See queued prompts while Codex is still running.
- Open completed tasks directly from notifications.
- Browse archived threads instead of keeping the active list crowded.
- Wrap long diff lines when reading on a narrow screen.
- Use Face ID or the device passcode to protect Codex separately.
- See skills and plugins inline in the composer before sending.
- Capture a photo directly into a prompt when the real-world object is the context.

- Start a side chat from selected transcript text rather than re-explaining the question.

None of these changes the fundamental agent model. Together, they remove the friction that makes a remote workflow feel fragile.

## The larger lesson

The best mobile software does not shrink a desktop interface. It identifies the decisions that matter when you are away from the desk and makes those decisions fast, legible, and safe.

That is how I think about Codex Remote now. It is where I can choose the right environment, set the objective, redirect a run, answer an approval, inspect the result, and keep the whole queue of engineering work coherent.

The computer still does the work. The phone keeps me in control.